Universidade Federal de Minas Gerais

ELT127 -Automação em Tempo Real

Relatório Técnico – TP

Etapa 1 – Arquitetura, Processos e Sincronização

Gerenciamento de Frota Robótica para Logística

Lucca Rigueira Sant'ana Nogueira – 2023038957
Matheus Muniz Soares – 2023038892
Pedro Henrique Chalfun de Aquino – 2023039066

Prof. Leandro Freitas

Belo Horizonte, 25 de outubro de 2026

Sumário

1 Introdução e objetivos da etapa **2**

2 Arquitetura do sistema **3**

3 Implementação dos processos **4**

3.1 Processo principal e coordenação sequencial 4

3.2 Criação dinâmica dos processos dos robôs 5

3.3 O que permanece isolado entre processos 5

4 Implementação das threads **5**

4.1 Thread de comando de navegação 5

4.2 Thread de status e tratamento de sensores 5

4.3 Thread do coletor de dados 5

5 Comunicação entre processos e troca de dados **6**

5.1 Canal IPC e enquadramento das mensagens 6

5.2 Sequência de uma operação de transporte 6

6 Mecanismos de sincronização **6**

6.1 Mutexes: exclusão mútua dentro do robô 6

6.2 Variáveis de condição e execução periódica 7

6.3 Exclusão mútua e espera limitada no IPC 7

6.4 Sincronização lógica de equipes e passagens 7

6.5 Persistência assíncrona e tratamento de falhas 8

7 Persistência e rastreabilidade **8**

8 Testes e evidências de troca de dados **9**

8.1 Comunicação IPC e processos 9

8.2 Navegação, logística e sincronização 9

8.3 Coleta e persistência de dados 10

8.4 Testes de integração 10

9 Resultados da etapa e limitações **13**

10 Compilação e reprodução **14**

11 Conclusão **14**

1 Introdução e objetivos da etapa

O transporte de cargas em galpões logísticos automatizados pode exigir a atuação coordenada de diferentes robôs móveis. Nesse contexto, cargas de tamanhos distintos são disponibilizadas em baias e precisam ser encaminhadas às zonas de entrega. Algumas operações podem ser realizadas por um único robô, enquanto outras exigem equipes de até quatro integrantes. Além da escolha dos destinos, o sistema deve considerar o deslocamento pelo mapa, a ocupação de passagens estreitas, o estado da bateria e a ocorrência de falhas.

A Figura 1 apresenta uma representação gráfica do ambiente logístico, que foi representado de forma matricial na abordagem adotada pelo grupo na implementação do projeto.

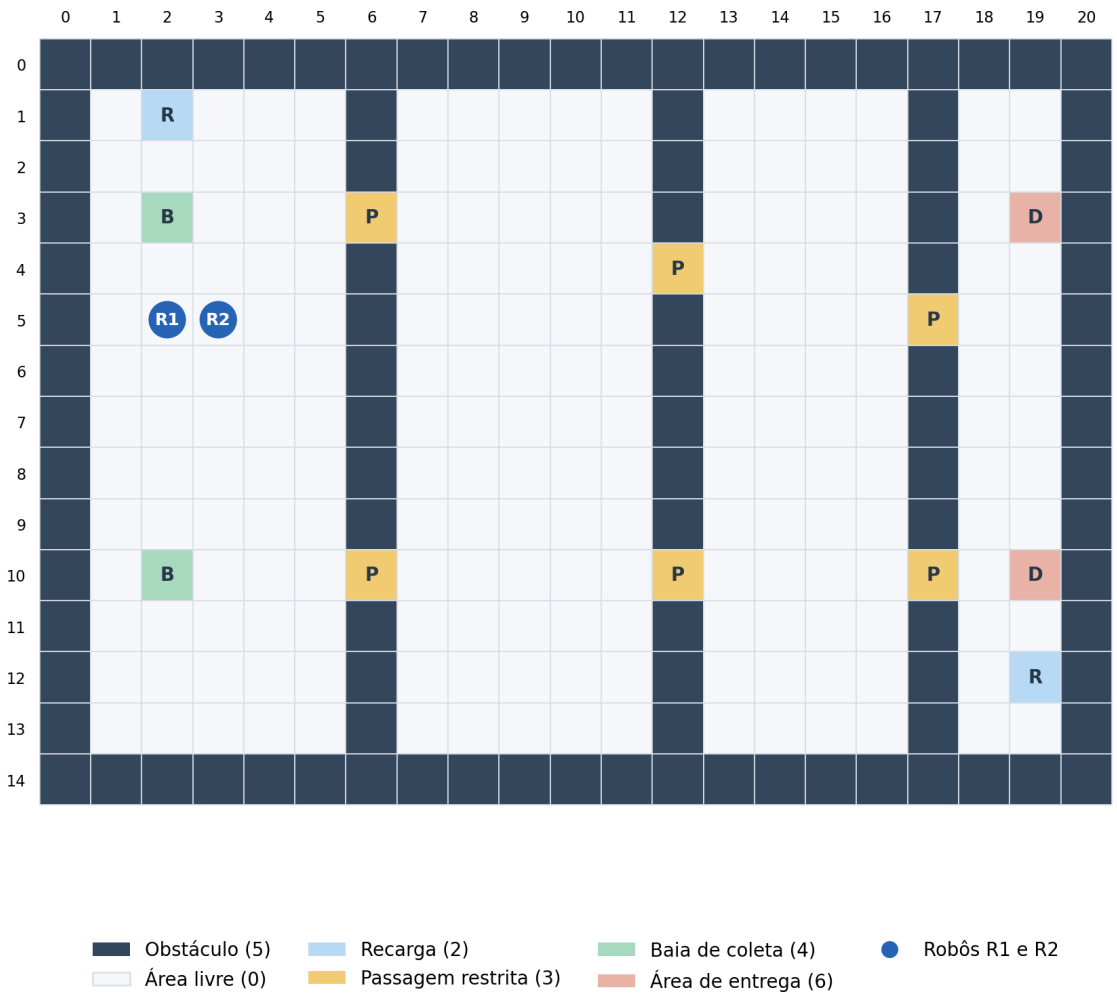


Figura 1: Representação gráfica do ambiente logístico

Os números entre parênteses na legenda correspondem aos códigos internos dos tipos de célula do mapa, não à capacidade de cada passagem restrita.

O objetivo geral do projeto é desenvolver um sistema de planejamento e simulação de uma frota robótica, com processos independentes para os agentes e mecanismos de comunicação e sincronização que permitam realizar missões individuais e colaborativas. Nesta primeira etapa, concentramos a implementação na arquitetura de execução, na troca de mensagens, na coordenação logística e na persistência dos eventos.

Como objetivos específicos desta entrega, definimos a criação dinâmica dos processos dos robôs, a execução das threads de navegação e status, a comunicação IPC com o processo central, o controle de equipes e passagens restritas e a gravação assíncrona de eventos em SQLite. Também desenvolvemos testes unitários e de integração para verificar o comportamento dos componentes e suas interações. As interfaces gráficas e a comunicação MQTT permanecem previstas para a segunda etapa.

2 Arquitetura do sistema

Organizamos a arquitetura em três partes: coordenação logística, execução dos robôs e persistência de dados em SQLite. O processo central gerencia missões e recursos simulados, enquanto cada robô executa em um processo independente.

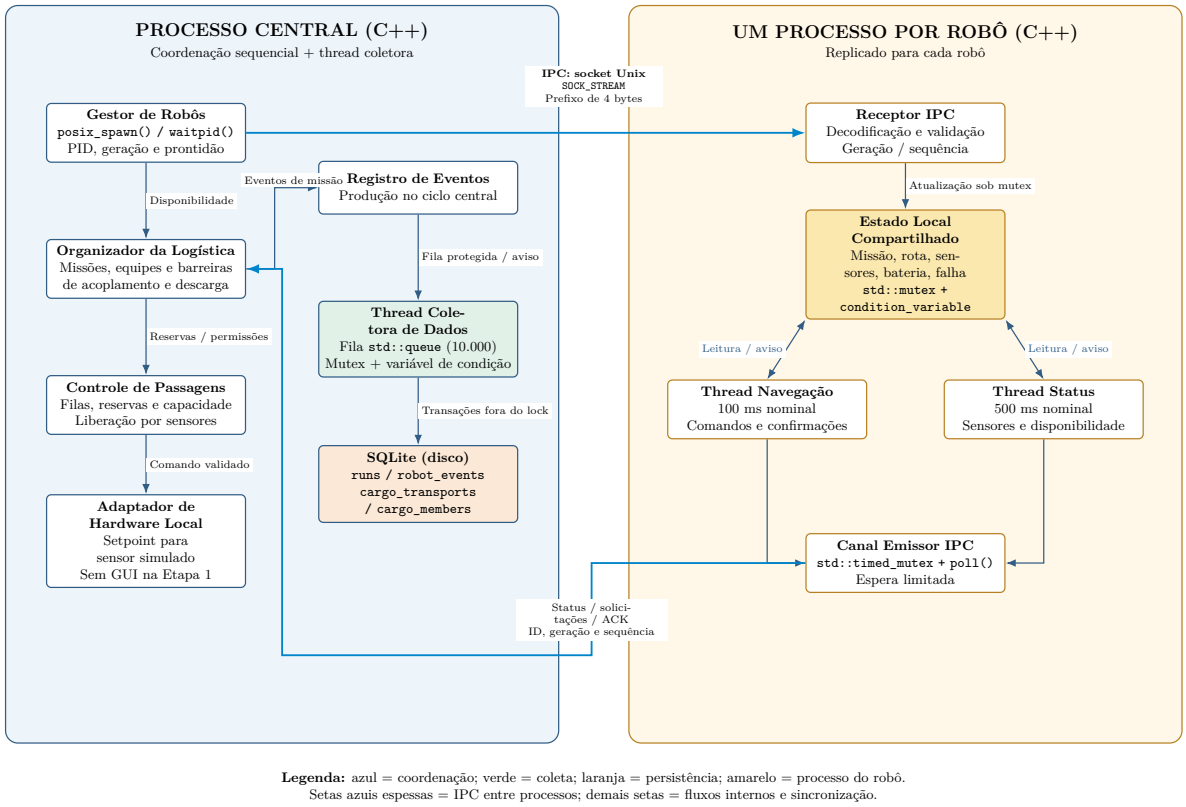


Figura 2: Arquitetura detalhada da Etapa 1: processos, threads, canais IPC e sincronização. Os módulos de coordenação atuam sequencialmente no ciclo central; as threads dos agentes e do coletor executam concorrentemente.

A Figura 2 delimita os processos e identifica as operações concorrentes. Cada robô possui um processo próprio, com receptor IPC, thread de navegação e thread de status. A troca entre o processo central e cada agente ocorre por socket Unix `SOCK_STREAM`, com prefixo de tamanho de quatro bytes e validação de identificador, geração e sequência. No processo do robô, o estado local é protegido por `std::mutex`; notificações por `std::condition_variable` coordenam despertares periódicos ou antecipados, e o envio compartilhado pelo socket é serializado com `std::timed_mutex`, usando `poll()` e limites de espera.

No processo central, o gestor, o organizador, o controle de passagens e o adaptador de hardware

são chamados pelo laço principal sequencialmente, e não constituem quatro threads independentes. O transporte colaborativo utiliza confirmações dos integrantes como barreira lógica, enquanto as passagens restritas são administradas por filas e reservas de capacidade, liberadas conforme leituras dos sensores; não são utilizados semáforos POSIX nesse controle. O registro de eventos alimenta uma fila `std::queue` protegida por mutex e variável de condição; uma thread coletora independente consome os eventos e grava em SQLite fora do bloqueio da fila. O `BufferEventos` circular, descrito na Seção 7, é um componente auxiliar testado, não a fila principal de persistência.

Optamos por utilizar o SQLite em vez de arquivos de log convencionais para garantir maior organização, integridade e rastreabilidade dos dados gerados durante a execução do sistema. O uso de um banco de dados relacional permite armazenar informações estruturadas sobre os robôs, eventos, missões e cargas transportadas, facilitando consultas e análises posteriores. Além disso, o SQLite oferece suporte a transações, restrições de integridade e recuperação em caso de falhas, reduzindo o risco de inconsistências nos registros. Por ser leve, não exigir um servidor dedicado e permitir integração direta com C++, mostrou-se uma solução adequada para o projeto. Associado à thread de coleta de dados, possibilita ainda a persistência assíncrona dos eventos, evitando que as operações de gravação bloqueiem diretamente o processamento logístico.

A Tabela 1 resume as responsabilidades dos componentes.

Tabela 1: Responsabilidades dos componentes implementados.

Módulo	Contexto	Responsabilidade
`AplicacaoCentral`	Processo central; laço principal	Processa mensagens, aciona logística, hardware e passagens.
`GestorRobos`	Mesmo processo central	Cria/encerra processos, acompanha PIDs, gerações, mensagens e prontidão.
`AgenteRobo`	Processo individual	Compartilha estado local entre threads de navegação e status.
`OrganizadorLogistica`	Ciclo central	Reserva equipes, define missões, acompanha fases e retomadas.
`ControlePassagens`	Ciclo central	Mantém fila, capacidade, ocupação e permissões de tráfego.
`AdaptadorHardwareLocal`	Ciclo central	Converte setpoints válidos em novas leituras de sensores.
`ColetorDados`	Thread própria	Consome eventos da fila e persiste registros no SQLite.

3 Implementação dos processos

3.1 Processo principal e coordenação sequencial

O executável `simulador_exec` permite selecionar cenários, banco de dados e modo interativo. A função principal inicia a aplicação, executa seus ciclos e coordena o encerramento. No modo automático, cada cenário possui limite de execução de 120 segundos.

O ciclo principal recebe mensagens IPC, atualiza sensores e permissões e processa as missões após eventos ou, em períodos de inatividade, a cada 200 ms aproximadamente. Os componentes de logística, passagens e hardware são executados sequencialmente. Com isso, concentramos o acesso ao estado central em um único fluxo de execução, reduzindo a necessidade de bloqueios.

3.2 Criação dinâmica dos processos dos robôs

Ao adicionar um robô, o gestor valida seu identificador, cria um par de sockets Unix e inicia um novo processo com `posix_spawn()`. O processo central mantém o PID, a identificação lógica, a geração e o canal de comunicação de cada agente.

A inicialização compreende a abertura do canal, a criação do processo e a confirmação de prontidão. O gestor só envia tarefas após receber `RoboPronto`. O número de geração permite distinguir reinicializações de um mesmo robô e descartar mensagens de instâncias anteriores.

No encerramento, o gestor solicita a parada e monitora o processo com `waitpid()`. Caso necessário, envia `SIGTERM` após dois segundos e `SIGKILL` após três. Desconexões inesperadas tornam o robô indisponível, sem reinicialização automática.

3.3 O que permanece isolado entre processos

Não há memória compartilhada entre o robô e o gestor. As informações são transmitidas por mensagens serializadas, cuja consistência depende da validação de geração, sequência, confirmações e leituras de sensores.

4 Implementação das threads

4.1 Thread de comando de navegação

Cada robô possui uma thread de navegação com período nominal de **100 ms**. Ela verifica a missão, o nível de bateria, possíveis falhas e confirmações pendentes. Havendo um próximo destino, solicita acesso à passagem antes de enviar o comando de movimento. A rota só avança quando o sensor confirma a posição esperada.

Ao concluir o percurso, o agente passa à fase correspondente da missão, como acoplamento, descarga ou recarga. Esse procedimento evita divergências entre posição estimada e posição confirmada.

4.2 Thread de status e tratamento de sensores

A thread de status possui período nominal de **500 ms**, com ativação antecipada diante de alterações de estado ou novos sensores. Ela processa leituras, acumula a distância percorrida, atualiza a disponibilidade e publica o estado do robô.

Os registros incluem identidade, missão, fase, posição, bateria, falhas e distância. Leituras antigas ou incompatíveis com a missão são descartadas; leituras intermediárias são consideradas no cálculo do deslocamento.

4.3 Thread do coletor de dados

O coletor utiliza uma thread independente para retirar eventos de uma fila sincronizada e gravá-los no SQLite. Dessa forma, as operações logísticas não precisam aguardar cada escrita em disco.

No encerramento, o coletor processa os eventos pendentes antes de fechar o banco. Erros de persistência são registrados e podem ser encaminhados ao processo central.

5 Comunicação entre processos e troca de dados

5.1 Canal IPC e enquadramento das mensagens

Os processos se comunicam por sockets Unix do tipo `SOCK_STREAM`. Como o fluxo não delimita mensagens, cada quadro contém um prefixo de quatro bytes que informa seu tamanho. O receptor recompõe mensagens fragmentadas, respeitando o limite de 64 KiB.

O protocolo transmite identificação, geração, sequência, comandos, missões e dados operacionais. A decodificação valida formato, versão e limites dos campos. O envio utiliza `poll()`, com prazo de até 1 s; o gestor aplica limites de 2 s para prontidão e 3 s para comunicação ou confirmação.

5.2 Sequência de uma operação de transporte

A Figura 3 apresenta a sequência de autorização, movimento e confirmação por sensor.

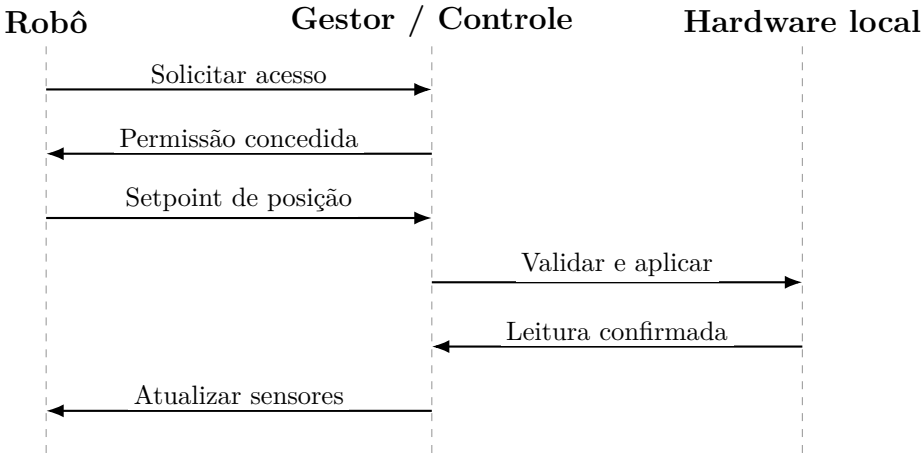


Figura 3: Fluxo de confirmação de um passo de navegação.

Na atribuição de missões, cada comando recebe uma confirmação associada ao seu identificador. Também são verificados geração, sequência e revisão da missão, evitando a aceitação de informações desatualizadas.

6 Mecanismos de sincronização

6.1 Mutexes: exclusão mútua dentro do robô

Utilizamos `std::mutex` para proteger o estado, a rota e as leituras de sensores compartilhadas entre as threads de navegação, status e o receptor de mensagens. Essa proteção evita alterações simultâneas que poderiam comprometer a consistência das informações utilizadas na movimentação.

Os bloqueios são gerenciados por `std::lock_guard` e `std::unique_lock`, garantindo a liberação dos mutexes ao final de cada operação. Também adotamos mutexes específicos para controlar a inicialização e o encerramento das threads, além de proteger a sequência das mensagens enviadas pelo IPC.

Essa separação permite proteger recursos distintos sem utilizar um único bloqueio para todas as operações do robô, reduzindo a interferência entre tarefas concorrentes.

6.2 Variáveis de condição e execução periódica

Utilizamos `std::condition_variable` para suspender as threads enquanto aguardam comandos, leituras ou encerramento, evitando consultas contínuas ao estado compartilhado.

As rotinas de navegação e status possuem períodos nominais de 100 ms e 500 ms, respectivamente, utilizando `steady_clock` e `wait_until()`. A thread de status também pode despertar antecipadamente diante de novas leituras ou alterações de estado.

Combinamos, assim, execução periódica e resposta a eventos, reduzindo processamento desnecessário sem depender exclusivamente da próxima atualização periódica.

No coletor de dados, outra variável de condição sinaliza a chegada de eventos à fila. O mutex é liberado antes das operações SQLite, permitindo que novas informações sejam recebidas enquanto ocorre a gravação em disco.

6.3 Exclusão mútua e espera limitada no IPC

No canal IPC, implementamos `std::timed_mutex` para impedir que diferentes threads intercalem bytes durante a transmissão de mensagens pelo mesmo socket.

Diferentemente de um bloqueio sem limite de espera, utilizamos `try_lock_until()` para estabelecer um prazo de aquisição do mutex. O envio também considera esse prazo e utiliza `poll()` para verificar a disponibilidade do canal.

Essa solução permite detectar quando uma transmissão não pode ser concluída dentro do tempo previsto, em vez de manter a thread bloqueada indefinidamente. Trata-se de uma escolha importante para a responsividade do sistema, pois facilita a identificação de atrasos e falhas de comunicação.

Também utilizamos identificadores de geração, sequência e comando para validar as mensagens recebidas. Dessa forma, evitamos que informações antigas sejam utilizadas após alterações de missão ou reinicializações dos processos.

6.4 Sincronização lógica de equipes e passagens

No transporte colaborativo, implementamos uma barreira lógica: a missão só avança quando todos os integrantes confirmam o acoplamento. O movimento coletivo exige propostas compatíveis de posição e destino, enquanto a descarga depende da confirmação de desacoplamento da equipe.

Para controlar passagens restritas, desenvolvemos um sistema de filas e reservas que considera a capacidade disponível, o sentido do deslocamento e a ocupação atual.

- **Controle por filas:** as solicitações são organizadas por passagem e avaliadas conforme a disponibilidade de vagas. Quando não há capacidade suficiente ou existe tráfego em sentido contrário, a solicitação permanece aguardando.
- **Reservas coletivas:** equipes acopladas reservam conjuntamente a quantidade de vagas necessária aos seus integrantes. Isso evita autorizar um deslocamento coletivo sem capacidade suficiente para a equipe.
- **Liberação por sensores:** as vagas somente são liberadas após a confirmação de saída dos robôs. Caso a posição seja desconhecida, não presumimos que o corredor esteja livre, evitando novas permissões baseadas em informações incompletas.

Como o gerenciamento das passagens ocorre exclusivamente no ciclo central, implementamos esse

controle por filas e permissões, sem necessidade de semáforos POSIX. Essa organização evita acessos concorrentes às estruturas de ocupação e concentra as decisões sobre os recursos compartilhados.

6.5 Persistência assíncrona e tratamento de falhas

Para registrar os eventos operacionais, utilizamos uma fila protegida por mutex e uma thread independente responsável pela persistência no SQLite.

Essa abordagem desacopla a produção dos eventos das operações de escrita em disco, evitando que cada registro interrompa diretamente o processamento logístico. Também estabelecemos um limite de 10.000 eventos na fila, permitindo identificar situações em que a produção supera a capacidade de armazenamento.

Durante o encerramento, a thread coletora processa os eventos pendentes antes de fechar o banco. Além disso, falhas de gravação são armazenadas e podem ser propagadas à aplicação central.

Esses mecanismos contribuem para a consistência dos registros e para o tratamento de falhas sem depender de gravações síncronas a cada evento.

Tabela 2: Mecanismos de sincronização utilizados no sistema.

Recurso	**Mecanismo**	**Finalidade**
Estado, rota e sensores	`std::mutex`	Garantir consistência entre navegação, status e recepção de mensagens.
Execução das threads	`condition_variable` e `wait_until()`	Coordenar ativações periódicas e notificações por eventos.
Envio de mensagens IPC	`std::timed_mutex` e `poll()`	Serializar mensagens e limitar a espera pelo canal.
Ciclo de vida dos agentes	Mutex, `notify_all()` e `join()`	Coordenar inicialização e encerramento das threads.
Passagens restritas	Filas e reservas	Controlar capacidade, sentido e ocupação dos corredores.
Transporte colaborativo	Barreira lógica e confirmações	Sincronizar acoplamento, deslocamento e descarga.
Persistência SQLite	Fila, mutex e thread coletora	Registrar eventos sem bloquear diretamente o ciclo logístico durante a escrita.

As soluções adotadas combinam exclusão mútua, sincronização por eventos, comunicação com espera limitada e controle lógico de recursos. Com isso, buscamos reduzir bloqueios desnecessários e manter a consistência das operações concorrentes, aspectos importantes para a Automação em Tempo Real.

7 Persistência e rastreabilidade

Os eventos operacionais são encaminhados ao coletor e armazenados em quatro tabelas SQLite: `runs`, `robot_events`, `cargo_transports` e `cargo_members`. Os registros permitem relacionar execuções, robôs, missões, cargas e integrantes das equipes.

Cada evento é gravado em transação, com reversão em caso de falha. Restrições de identidade evitam duplicações e os registros utilizam horários Unix em milissegundos. Os prazos operacionais são medidos separadamente por relógio monotônico.

Também implementamos o `BufferEventos`, que mantém um histórico circular limitado, com capacidade padrão de 100 eventos. Esse buffer utiliza `std::deque` e `std::mutex`; ao atingir a capacidade, os registros mais antigos são substituídos pelos novos. O componente é exercitado em testes e no programa auxiliar, mas não constitui a fila obrigatória do fluxo principal. Nesse fluxo, os eventos são encaminhados diretamente ao `ColetorDados`.

A fila interna do coletor utiliza `std::queue`, mutex e variável de condição, com limite de 10.000 eventos pendentes. A thread consumidora aguarda a chegada de registros, retira cada evento sob proteção do mutex e realiza a gravação no SQLite depois de liberar a fila. Assim, a produção de novos eventos não fica bloqueada durante toda a operação de escrita. Ao atingir 10.000 eventos pendentes, a tentativa de enfileirar outro registro lança uma exceção do tipo `std::runtime_error`; não há espera automática por espaço nem substituição silenciosa de registros. Esse comportamento difere do descarte circular empregado pelo `BufferEventos` e deverá ser considerado em testes de carga.

8 Testes e evidências de troca de dados

Implementamos uma suíte de testes automatizados utilizando GoogleTest, com o objetivo de verificar o funcionamento dos módulos e a comunicação entre eles. Os testes foram organizados para avaliar tanto o comportamento individual dos componentes quanto sua integração, com atenção especial aos mecanismos de sincronização e à consistência dos dados.

Esses testes se encontram no repositório, sendo possível executá-los localmente com o CTest, que apresenta os resultados diretamente no terminal. A seguir explicaremos a organização dos testes e os principais aspectos verificados em cada grupo.

8.1 Comunicação IPC e processos

Implementamos 8 testes para o protocolo IPC e o estado compartilhado da frota. Eles verificam a serialização e recuperação das mensagens, a recepção de dados fragmentados, a rejeição de mensagens inválidas e o tratamento de informações antigas ou duplicadas.

Esses testes permitem verificar a integridade das informações transmitidas entre processos, incluindo o controle por geração e sequência das mensagens.

8.2 Navegação, logística e sincronização

Desenvolvemos 18 testes distribuídos entre os seguintes módulos:

- **Agente do robô (7 testes):** verificam permissões de movimentação, confirmação por sensores, tratamento de falhas, recarga e inicialização e encerramento das threads.
- **Organizador logístico (4 testes):** avaliam a formação de equipes, a espera pelos integrantes, o cancelamento de missões e a retomada após interrupções.
- **Passagens e hardware (4 testes):** verificam limites de ocupação, filas de espera, reservas coletivas e sincronização do movimento entre integrantes da equipe.
- **Mapa e rotas (3 testes):** avaliam a validade do mapa, o planejamento de trajetos com obstáculos e as restrições de capacidade dos caminhos.

Com esses testes, buscamos verificar se as decisões de movimentação respeitam as condições de segurança e sincronização estabelecidas para os robôs.

8.3 Coleta e persistência de dados

Implementamos 13 testes relacionados ao armazenamento das informações:

- **Coletor de dados (7 testes):** verificam a gravação de eventos e transportes no SQLite, o processamento de eventos pendentes, a concorrência entre threads produtoras e o comportamento durante inicialização e encerramento.
- **Buffer de eventos (4 testes):** avaliam a retenção do histórico circular, a escrita concorrente, a persistência dos eventos e o tratamento de registros rejeitados.
- **Persistência e migração (2 testes):** verificam a preservação de registros existentes, o armazenamento dos integrantes de uma equipe e o tratamento de falhas de gravação.

Essas verificações são importantes para avaliar o funcionamento da fila sincronizada e da thread coletora, principalmente quando múltiplas threads produzem eventos simultaneamente. No teste `HandlesConcurrentProducersWithoutLosingEvents`, quatro threads produzem 50 eventos cada; após o encerramento do coletor, verificamos que os 200 eventos foram persistidos. O resultado registrado evidencia ausência de perda nesse cenário de concorrência, sem demonstrar ausência geral de condições de corrida.

Também desenvolvemos um programa auxiliar para gerar um banco SQLite persistente, permitindo consultar manualmente os registros após a execução e verificar as informações armazenadas.

8.4 Testes de integração

Além dos 39 testes unitários, implementamos 9 casos de integração: 8 execuções parametrizadas de cenários e um teste específico de encerramento.

Os cenários abrangem transporte básico e cooperativo, disponibilidade de equipes, passagens restritas, rotas coletivas, falhas, recarga e gerenciamento de processos. Verificamos a conclusão dos cenários, o encerramento dos processos dos robôs e a existência dos registros esperados no banco de dados.

O teste de encerramento verifica também a finalização dos processos ativos e o processamento dos eventos aceitos pelo coletor.

Ao todo, a suíte possui 48 casos de teste, considerando as instâncias parametrizadas de integração. Sua execução pode ser realizada pelo CTest, com os resultados apresentados diretamente no terminal.

Tabela 3: Resultados de integração registrados na pasta de evidências (25/10/2026).

Cenário	Eventos SQLite	Cargas transportadas
Básico	42	1
Cooperativo	593	3
Disponibilidade	94	1
Passagens restritas	287	6
Rota em equipe	181	1
Falhas	114	1
Recarga	48	1
Gerenciamento de processos	11	0
Total	**1.370**	**14**

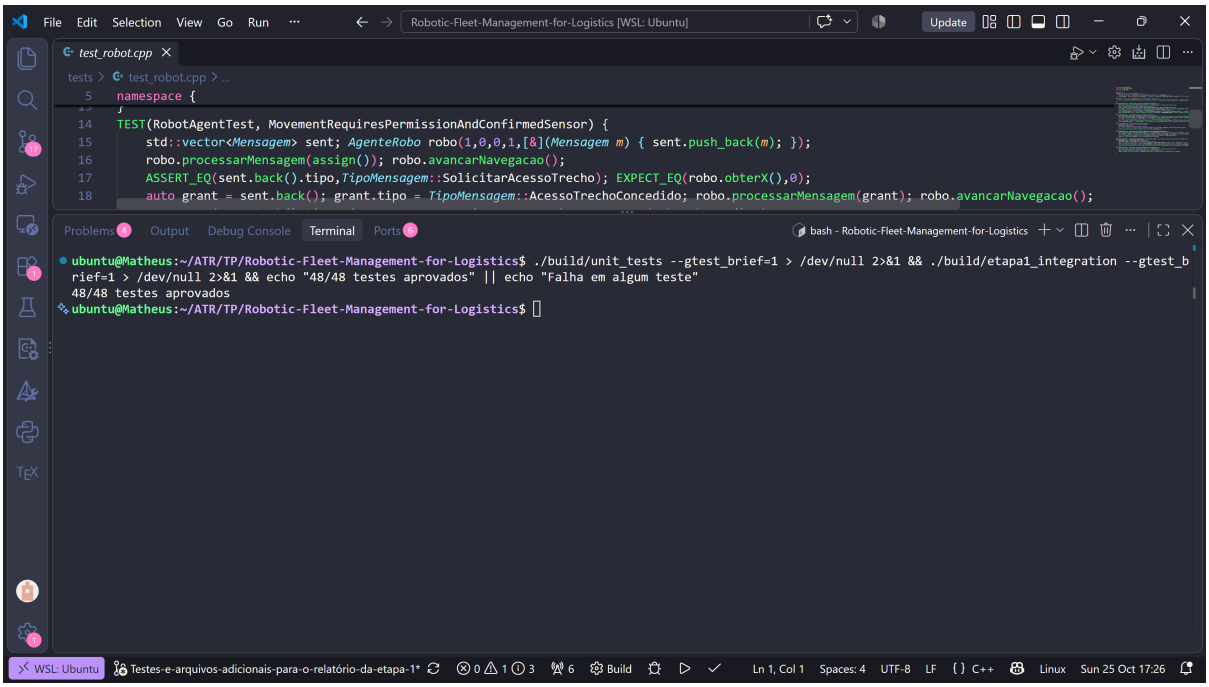


Figura 4: Script para verificação da execução dos 48 casos de teste, aprovação em todos.

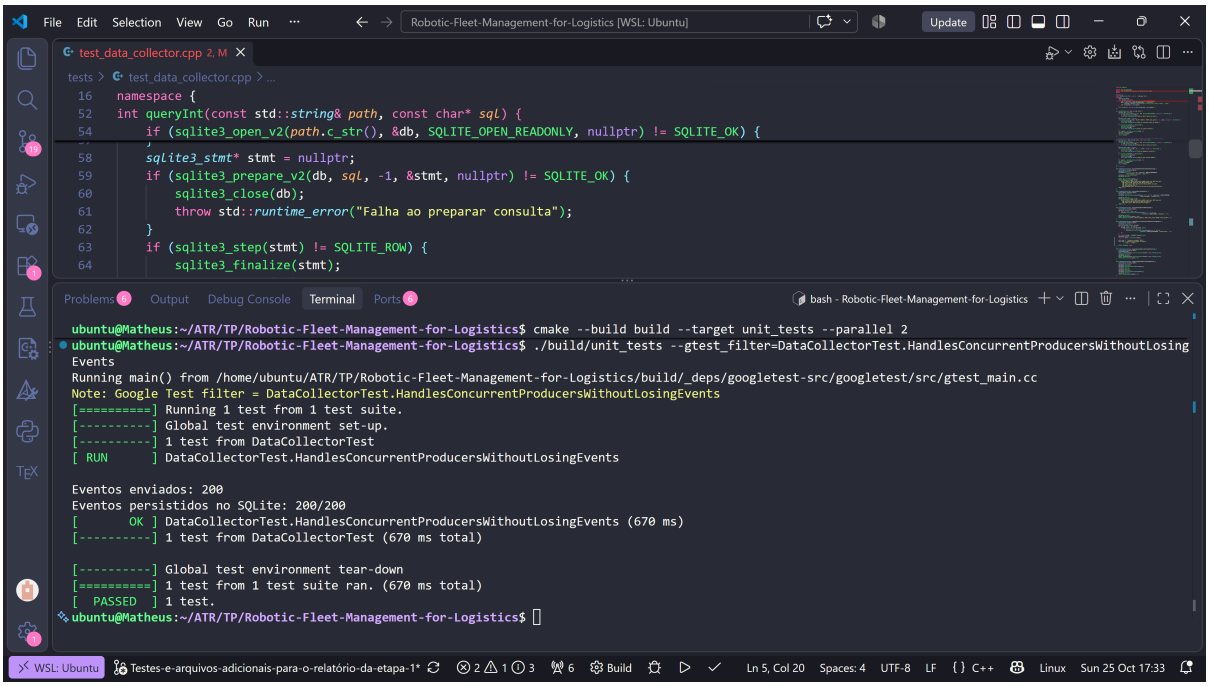


Figura 5: Teste de concorrência do ColetorDados, com quatro threads produtoras e confirmação da persistência de 200 eventos no banco SQLite.

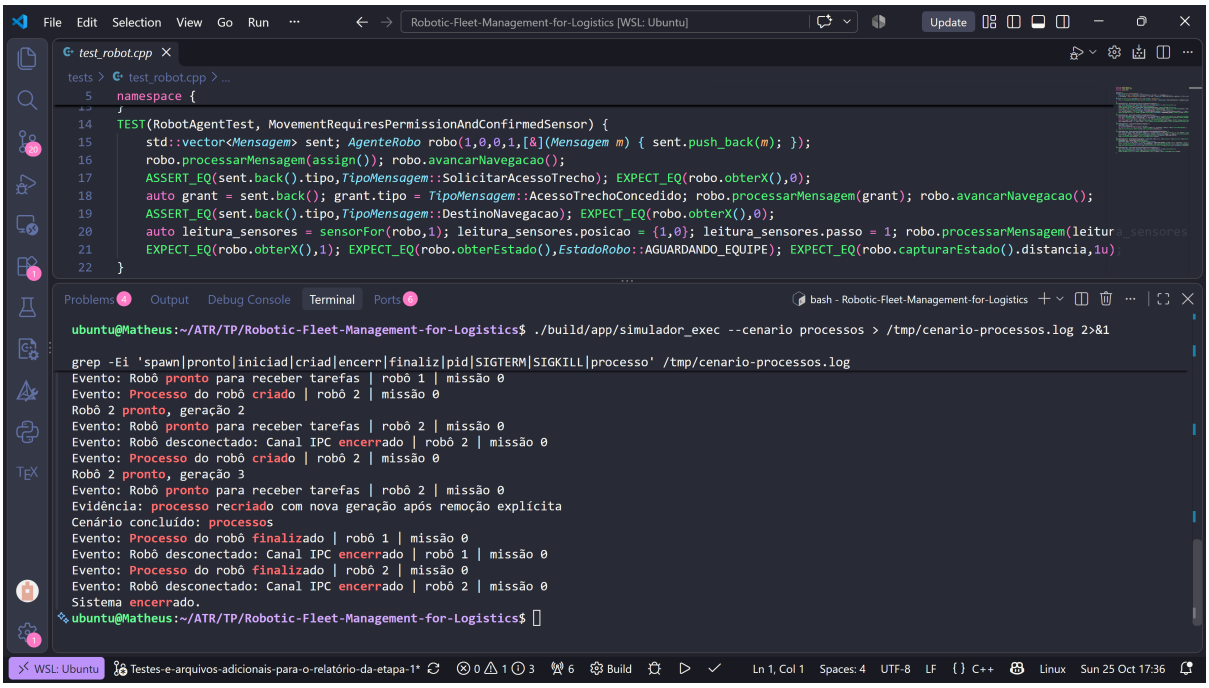


Figura 6: Evidência da criação dinâmica, confirmação de prontidão e encerramento dos processos dos robôs durante o cenário de gerenciamento.

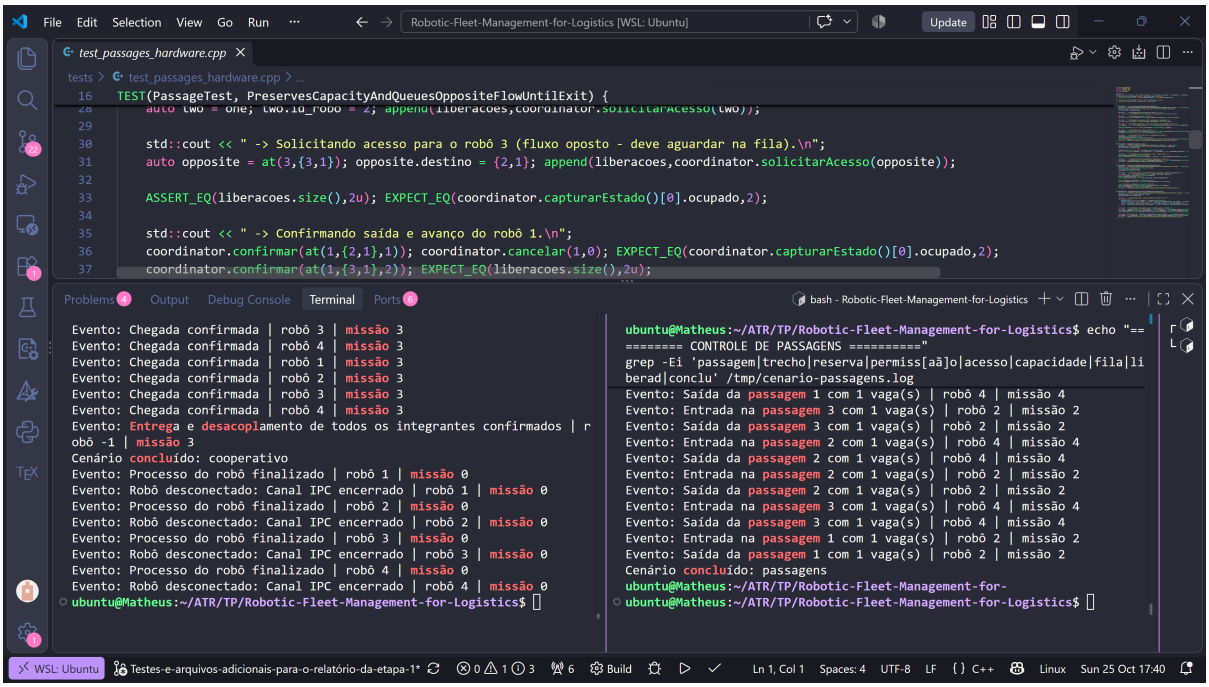


Figura 7: Evidências da execução do transporte cooperativo e do controle de passagens restritas, demonstrando a sincronização entre robôs, a coordenação das missões e o gerenciamento de recursos compartilhados.

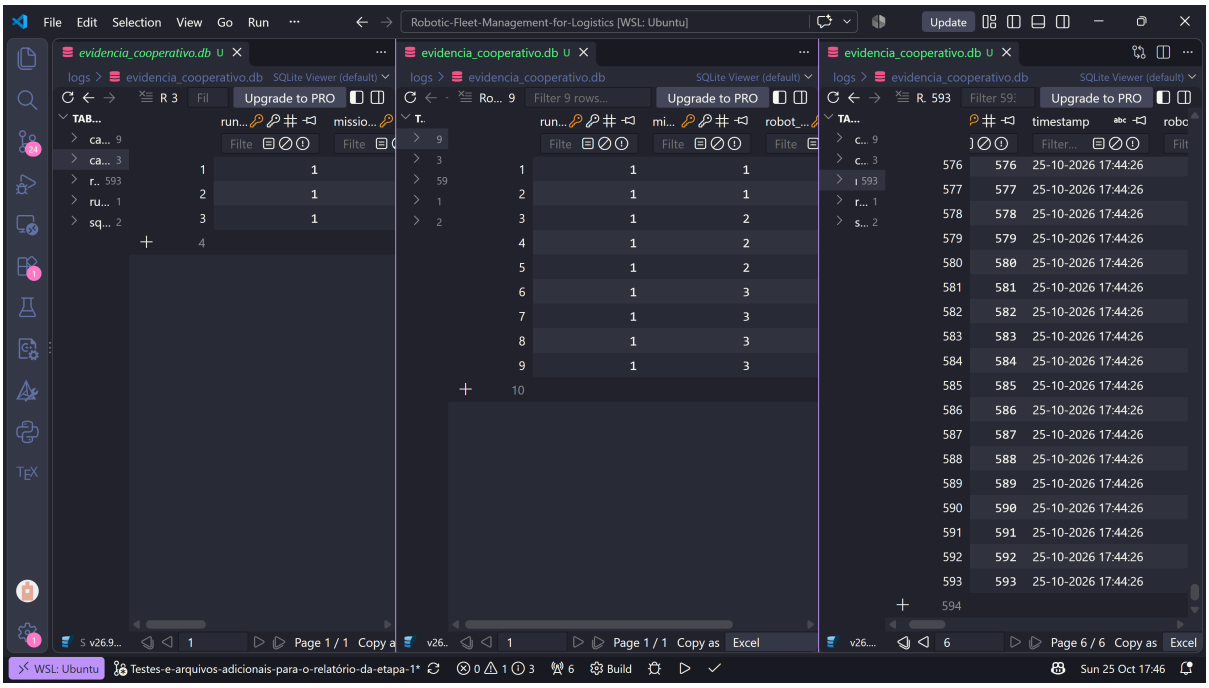


Figura 8: Inspeção do banco SQLite gerado pelo cenário cooperativo, apresentando os registros de três transportes de cargas, nove associações de integrantes às missões e 593 eventos operacionais persistidos, demonstrando a rastreabilidade das operações.

Os testes de concorrência e os cenários de integração fornecem evidência de integridade dos registros, progresso e término nas execuções observadas. A serialização de envios, a exclusão mútua local e a separação entre produtores e consumidor reduzem riscos de acesso concorrente indevido. Os testes em si, não garantem a ausência de todas as condições de corrida, bloqueio ou inanição, porém, conforme observado anteriormente, garantem a corretude nos casos de teste, que tentam abrangir as situações mais prováveis de ocorrerem durante a execução do sistema.

9 Resultados da etapa e limitações

Concluímos a primeira etapa com os principais componentes implementados, incluindo processos independentes para os robôs, threads de navegação e status, comunicação IPC, sincronização de equipes e persistência SQLite.

Também desenvolvemos uma suíte de 48 casos de teste para verificar os módulos e suas interações. A arquitetura adotada concentra o gerenciamento logístico no ciclo central e utiliza mutexes, variáveis de condição e filas nos pontos de concorrência, buscando reduzir bloqueios desnecessários e preservar a consistência das operações.

Para as próximas etapas, permanecem o desenvolvimento das interfaces gráficas, a integração MQTT e as avaliações de latência e escalabilidade. Os períodos de 100 ms e 500 ms definidos para as threads ainda são nominais, pois não realizamos medições de pior caso nem configuramos políticas de escalonamento de tempo real estrito.

10 Compilação e reprodução

A compilação e a execução dos cenários podem ser reproduzidas, a partir da raiz do projeto, pelos comandos:

```
cmake -S . -B build -DBUILD_TESTING=OFF
cmake --build build --parallel 2
./build/app/simulador_exec --cenario base
./build/app/simulador_exec --cenario disponibilidade
./build/app/simulador_exec --cenario processos
```

Para definir outro arquivo SQLite, utiliza-se `-banco logs/nome-da-execucao.db`. Com GoogleTest disponível, a suíte pode ser executada por:

```
cmake -S . -B build -DBUILD_TESTING=ON
cmake --build build --parallel 2
ctest --test-dir build --output-on-failure
```

Consulta SQL para verificar transportes e integrantes:

```
SELECT t.mission_id, t.cargo_id, t.status,
      m.robot_id
FROM cargo_transports AS t
JOIN cargo_members AS m
  ON m.run_id = t.run_id
 AND m.mission_id = t.mission_id
ORDER BY t.mission_id, m.robot_id;
```

11 Conclusão

Nesta primeira etapa, definimos e implementamos a arquitetura básica do sistema de gerenciamento de frota robótica. Organizamos a aplicação em um processo central e processos independentes para cada robô, com threads de navegação e status, comunicação por sockets Unix e sincronização por mutexes, variáveis de condição e confirmações de mensagens. O controle de passagens e a coordenação de equipes são realizados pelo ciclo central, enquanto a persistência dos eventos é executada de forma assíncrona pela thread coletora.

Desenvolvemos 39 testes unitários e 9 casos de integração para verificar a comunicação IPC, a consistência dos estados, o transporte colaborativo, as passagens restritas, a gravação de eventos e o encerramento dos processos. As execuções dos testes registraram aprovação dos 48 casos, término dos oito cenários e integridade dos bancos SQLite. Dessa forma, distinguimos a cobertura prevista nos testes implementados das conclusões efetivamente sustentadas pelas execuções documentadas.

Como próximos passos, pretendemos integrar a comunicação MQTT, desenvolver as interfaces de simulação e operação remota e ampliar os testes de funcionamento do sistema completo. Também serão necessárias medições de latência, variação dos períodos de execução, tempos de espera, resposta a falhas e escalabilidade com diferentes quantidades de robôs. Os períodos de 100 ms e 500 ms permanecem nominais, pois ainda não foram estabelecidas garantias de pior caso nem políticas de escalonamento de tempo real estrito.